

1.

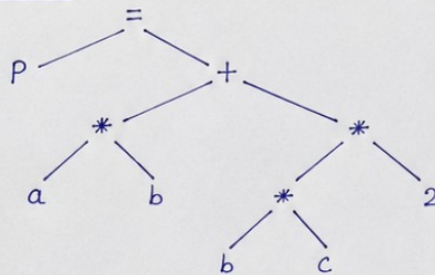
Trace the processing of the input expression :

$$P = (a * b) + (b * c) * 2$$

(a) Lexical Analysis (Tokens)

S.No	Lexeme	Token	S.No	Lexeme	Token
1	P	Identifier	9	(	Left Parenthesis
2	=	Assignment Operator	10	b	Identifier
3	(	Left Parenthesis	11	*	Multiplication Operator
4	a	Identifier	12	c	Identifier
5	*	Multiplication Operator	13	)	Right Parenthesis
6	b	Identifier	14	*	Multiplication Operator
7	)	Right Parenthesis	15	2	Integer Constant
8	+	Addition Operator	16	EOF	End of Input

(b) Syntax Tree (Parsing Phase)



(c) Intermediate Code (Three Address Code)

$t_1 = a * b$
 $t_2 = b * c$
 $t_3 = t_2 * 2$
 $t_4 = t_1 + t_3$
 $P = t_4$

(d) Optimized Intermediate Code

$t_1 = a * b$
 $t_2 = b * c$
 $t_2 = t_2 * 2$
 $P = t_1 + t_2$

(e) Final Target Code (Example Assembly Code)

LOAD R1, a
 MUL R1, b
 LOAD R2, b
 MUL R2, c
 MUL R2, #2
 ADD R1, R2
 STORE P, R1

Significance of Final Target Code :

- Converts intermediate code into machine/assembly instructions.
- Can be executed directly by the processor.
- Uses CPU registers efficiently.
- Produces faster execution with optimized instructions.
- Final output of the compiler before program execution.

2. Construct the language and write any FIVE valid strings for each Regular Expression.

a) $(aa^*)^*$

Language : Any string containing only a's (including empty string).

Five valid strings:

- (i) a
- (ii) aa
- (iii) aaa
- (iv) aaaa
- (v) aaaaa

b) $(b^*)^*abb$

Language : Any number of b's followed by abb.

Five valid strings:

- (i) abb
- (ii) babb
- (iii) bbabb
- (iv) bbbabb
- (v) bbbbabb

c) $(a^*b^*)^*aba(a^*+b^*)^*$

Language : Strings that contain the substring "aba".

Five valid strings:

- (i) aba
- (ii) abaa
- (iii) abab
- (iv) aabaa
- (v) babab

d) $(ab)^*$

Language : Zero or more repetitions of "ab".

Five valid strings:

- (i) ab
- (ii) abab
- (iii) ababab
- (iv) abababab
- (v) ababababab

e) $(aba)^* + (a^*b)^*$

Language : Strings formed either by repeating "aba" or by repeating the pattern a^*b .

Five valid strings:

- (i) aba
- (ii) abaaba
- (iii) b
- (iv) ab
- (v) aab

3. Find the matrix of tokens in the following C statement.

```
main ( )
{
    int a=16;
    char b='x';
    int t = 0 , s=3;
    t = t + a++;
    s = s - t ++t    m = 40.5;
}
```

Identify the Lexical errors in the following C statement &

```
void main()
{
    int x=10, y=20;
    char *;
    a = 4;
    s = 10ak
}
```

Solution :

(A) Matrix of tokens :

Lexeme	Token Name	Token Type
main	MAIN	Keyword
()	LPAREN	Symbol
()	RPAREN	Symbol
{	LBRACE	Symbol
int	INT	Keyword
a	ID	Identifier
=	ASSIGNOP	Operator
16	NUM	Constant
;	SEMICOLON	Symbol
char	CHAR	Keyword
b	ID	Identifier
=	ASSIGNOP	Operator
'x'	CHAR_CONST	Constant
;	SEMICOLON	Symbol
int	INT	Keyword
t	ID	Identifier
=	ASSIGNOP	Operator
0	NUM	Constant
,	COMMA	Symbol
s	ID	Identifier
=	ASSIGNOP	Operator
3	NUM	Constant
;	SEMICOLON	Symbol
t	ID	Identifier
=	ASSIGNOP	Operator
t	ID	Identifier
+	PLUS	Operator
a	ID	Identifier
++	INCREMENT	Operator
;	SEMICOLON	Symbol
s	ID	Identifier
=	ASSIGNOP	Operator
s	ID	Identifier
-	MINUS	Operator
t	ID	Identifier
+	PLUS	Operator
++	INCREMENT	Operator
t	ID	Identifier
m	ID	Identifier
=	ASSIGNOP	Operator
40.5	REAL_NUM	Constant
;	SEMICOLON	Symbol
}	RBRACE	Symbol

(B) Lexical errors in the given C statement :

1. char *; → Missing identifier after *.
2. a = 4; → 'a' is not declared.
3. s = 10ak → Invalid numeric constant (contains alphabet 'k').

Corrected version :

```
void main()
{
    int x = 10, y = 20;
    char *p;
    int a = 4;
    int s = 100;    // corrected constant
}
```

4. Write a regular expression corresponding to each of the following subsets of $\{1\}^*$.

(i) The language of all strings that have an even number of 1's.

Regular expression: $(11)^*$

Explanation: Every 11 contributes two 1's. Any number of pairs of 1's gives an even number of 1's (including empty string ϵ).

Examples: ϵ , 11, 1111, 111111,

(ii) The language of all strings that contain at least one 1.

Regular expression: $1(1)^*$

Explanation: The first 1 ensures at least one 1 is present.
 $(1)^*$ allows any number of additional 1's (or none).

Examples: 1, 11, 111, 1111,

(iii) The language of all strings in which both the number of 1's and the number of 0's are odd.

Regular expression: $11(11)^*$

Explanation: Starts with 11 (one 1 for each odd count) and $(11)^*$ adds pairs of 1's which keep both counts odd.

Examples: 11, 1111, 111111, 11111111,

(iv) The language of all strings that start with 1 and end with 0.

Regular expression: $1(1)^*0$

Explanation: Starts with 1, $(1)^*$ allows any number of 1's (including zero), and ends with 0.

Examples: 10, 110, 1110, 11110,

(v) The language of all strings that do not contain consecutive 1's.

Regular expression: $(\epsilon+0)(10)^*(\epsilon+1)$

Explanation: Strings are made of 0's and 1's in such a way that no two 1's appear together. After every 1, a 0 must follow.

$(\epsilon+0) \rightarrow$ allows starting with ϵ or 0

$(10)^* \rightarrow$ any number of 10 pairs

$(\epsilon+1) \rightarrow$ allows ending with ϵ or 1

Examples: ϵ , 0, 1, 01, 10, 101, 010, 1001,

5. Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens :

1. Keywords : if, else, while, return, int, float
2. Operators : +, -, *, /
3. Separators : (,), {, }, , , ;
4. Identifiers : A sequence of letters (followed by a sequence of digits, e.g., var123)
5. Integer literals : A sequence of digits, e.g., 12345
6. Floating point literals : A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

(A) Lexical analyzer codes to tokenize the following input string:

int var123 = 12345; float sum456 = var123 + 100.5; return;

Solution :

Lexeme	Token Name	Token Type
int	INT	Keyword
var123	ID	Identifier
=	ASSIGNOP	Operator
12345	NUM	Integer Literal
;	SEMI	Separator
float	FLOAT	Keyword
sum456	ID	Identifier
=	ASSIGNOP	Operator
var123	ID	Identifier
+	PLUS	Operator
100.5	REAL_NUM	Floating Point Literal
;	SEMI	Separator
return	RETURN	Keyword
;	SEMI	Separator

(B) Questions :

a) How many tokens are there in the given input string?

Ans: There are 14 tokens in the given input string.

b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

Ans:

- Keywords : int, float, return
- Operators : =, +
- Separators : ;
- Identifiers : var123, sum456
- Integer literals : 12345
- Floating-point literals : 100.5

c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of states required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

Ans: An estimate of FSM states required is about 20-25.

Reasoning :

- Identifiers : start → letter → (letter/digit)* → accept (need 4-5 states)
- Integer literals : start → digit → digit* → accept (3-4 states)
- Floating point literals : digits → '.' → digits → accept (4-5 states)
- Keywords, operators and separators : can be recognized with 1-2 states each.
- Start and final states are also included.

Hence, approximately 20 to 25 states are sufficient.